



Arquitectura de Microservicios - Proyecto VraKBen

Asignatura: Desarrollo Fullstack III

Sigla: DSY1106

integrantes: Ian Badilla / Vicente Placencia

1. Descripción del Contexto y Necesidades del Cliente

1.1. El Problema Operativo

A pesar de su éxito comercial, VraKBen enfrenta actualmente una grave crisis de gestión interna. La organización opera bajo un modelo de sistemas descentralizados y obsoletos —basados en planillas de cálculo y software antiguo— que carecen de comunicación entre sí. Esta fragmentación tecnológica impide una respuesta ágil a las demandas del mercado y genera ineficiencias que afectan directamente la rentabilidad y la reputación de la marca.

1.2. Puntos Críticos Identificados

Quiebres de Stock e Inventario Fantasma: Existe una desincronización crítica entre el taller de Desabolladura y Pintura (D&P) y la tienda. Debido a que el sistema no descuenta en tiempo real los insumos que los mecánicos retiran (como lijas o pintura), se produce el fenómeno de "inventario fantasma", donde se ofrecen y venden repuestos de forma online que físicamente ya no se encuentran en bodega.

Caos en el Agendamiento y Operación: El flujo de trabajo del taller se ve constantemente interrumpido. Aunque los clientes reservan horas para reparaciones específicas (ej. motor), el sistema no aparta previamente los repuestos necesarios. Al llegar el vehículo, la falta de insumos retrasa la operación y afecta la planificación diaria del personal técnico.

Falta de Trazabilidad del Cliente: No existe un historial unificado de las interacciones. Las áreas de ventas y el taller manejan la información en silos separados; si un cliente adquiere repuestos de carrocería e intenta agendar su instalación, el taller no tiene visibilidad inmediata de esa compra, lo que deriva en una experiencia de usuario deficiente y descoordinada.

Cuellos de Botella Tecnológicos: La arquitectura monolítica actual es altamente inestable. Durante periodos de alta demanda (como consultas masivas de stock de neumáticos), el sistema colapsa, impidiendo que los mecánicos ingresen sus órdenes de trabajo. Esto demuestra una falta crítica de escalabilidad y disponibilidad del sistema actual.

2. Selección y Justificación de Patrones de Arquitectura

2.1. Patrón de Descomposición por Capacidades de Negocio

Se han segmentado las responsabilidades en 4 áreas lógicas para garantizar que un pico de demanda en una (ej. Ventas) no colapse la otra (ej. Operación del Taller):

- **Gestión de Inventario (Núcleo Logístico):** Incluye `ms-stock-engine`, `ms-catalog`, `ms-warehouse-location` y `ms-supplier-procurement`. Este bloque elimina el "Inventario Fantasma" al centralizar la verdad del stock.
- **Operación del Taller (D&P y Mecánica):** Compuesto por `ms-appointment-scheduler`, `ms-resource-planner`, `ms-job-orders` y `ms-vehicle-history`. Optimiza el flujo de trabajo técnico y la trazabilidad.
- **Ventas y Clientes:** Integra `ms-customer-profile`, `ms-shopping-cart` y `ms-order-management`. Mejora la experiencia de usuario y la consistencia transaccional.
- **Infraestructura de Soporte:** `ms-api-gateway`, `ms-config-server`, `ms-auth-server` y `ms-notification`.

2.2. Patrones de Comunicación y Resiliencia

- **API Gateway (Spring Cloud Gateway):** Actúa como el "punto de entrada único", resolviendo los cuellos de botella al gestionar el tráfico de forma eficiente y redirigir las peticiones a los servicios correspondientes.
- **Database per Service (PostgreSQL):** Cada microservicio gestiona su propia persistencia. Esto evita que una consulta masiva a la base de datos de neumáticos bloquee el ingreso de órdenes de trabajo de los mecánicos.
- **Externalized Configuration (Config Server):** Centraliza los parámetros de los 14 servicios, facilitando el mantenimiento y la escalabilidad.

3. Justificación de Herramientas y Estrategias de Implementación

Spring Boot 3 & Java 17: Proporcionan un entorno robusto y tipado para el desarrollo rápido de servicios web.

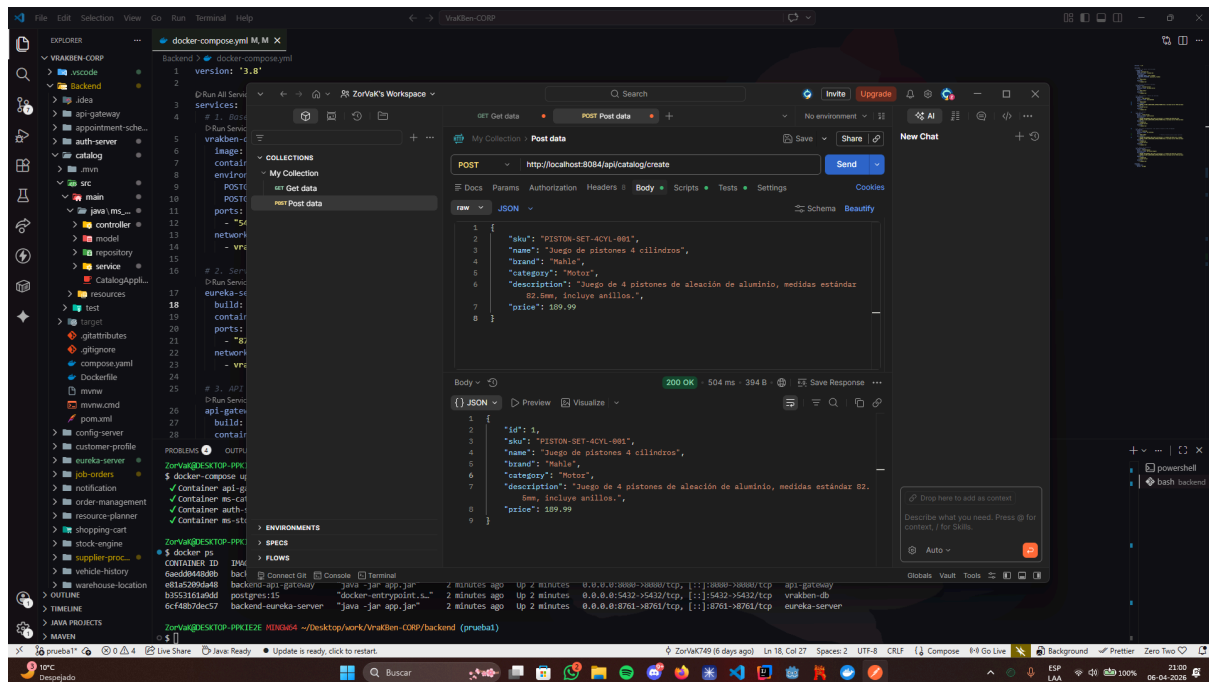
Docker & Docker Compose: Se utiliza la estrategia de **Multi-stage Build** en los **Dockerfile**. Esto permite que la compilación (vía Maven) y la ejecución ocurran en contenedores aislados, eliminando conflictos de versiones locales y garantizando que el sistema funcione idénticamente en cualquier entorno.

OpenFeign: Utilizado para la comunicación síncrona entre servicios. Por ejemplo, permite que **order-management** descuente stock automáticamente en el **stock-engine** tras una venta exitosa.

PostgreSQL: Motor relacional sólido para garantizar la consistencia de datos críticos como transacciones financieras y registros de vehículos.

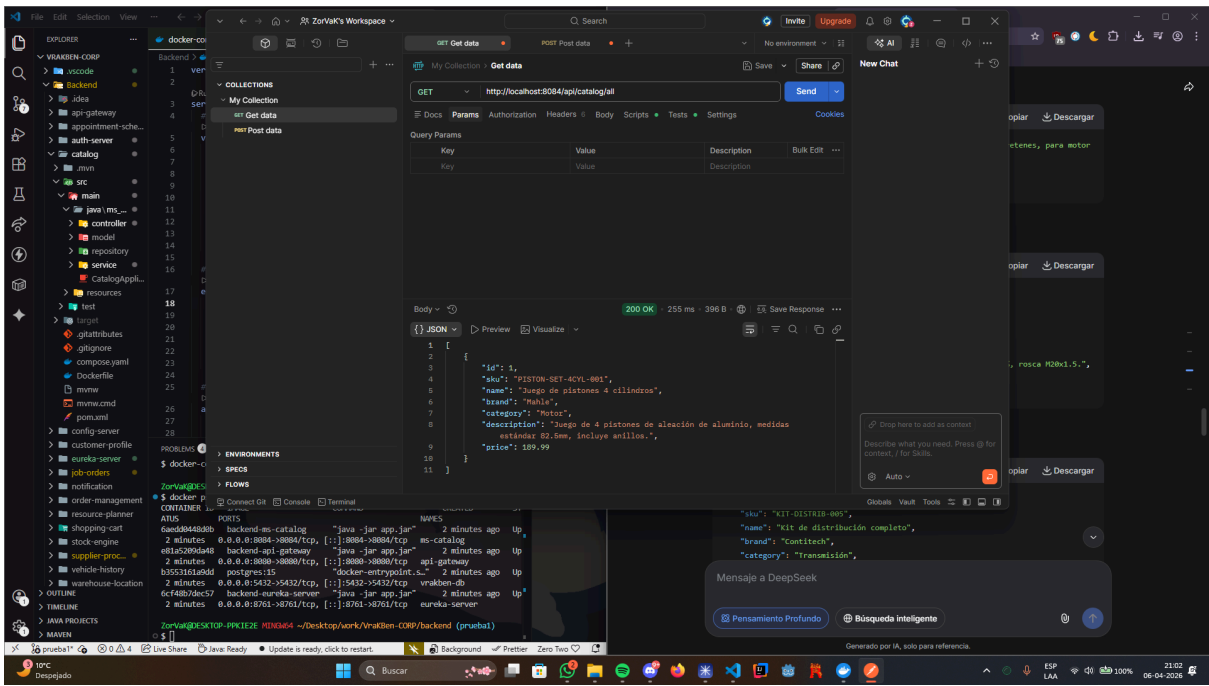
muestra de las conexiones de los microservicios

POST N° figura, Servicio-Catalog



Nota: Se desarrolló un Post de el servicio de catalogo con un “repuesto” ingresado en el sistema, con el fin de probar el funcionamiento correcto del microservicio creado.

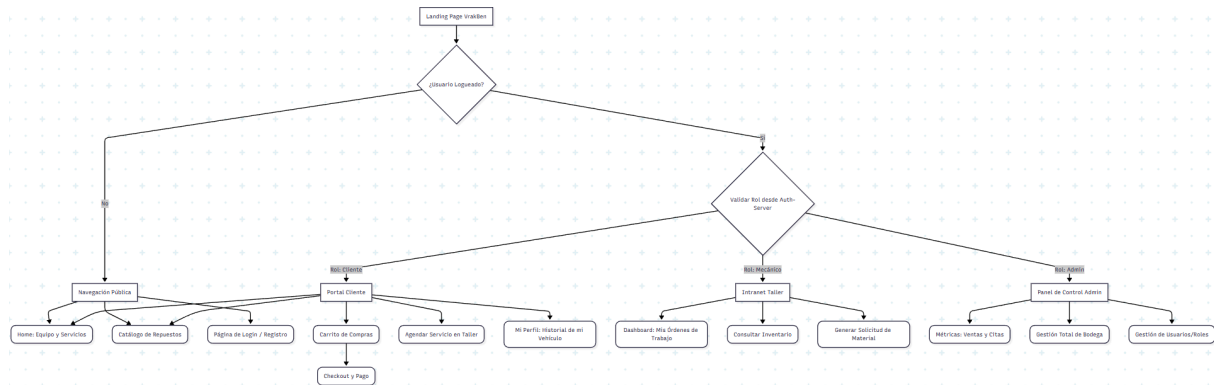
GET N° Figura



Nota: GET

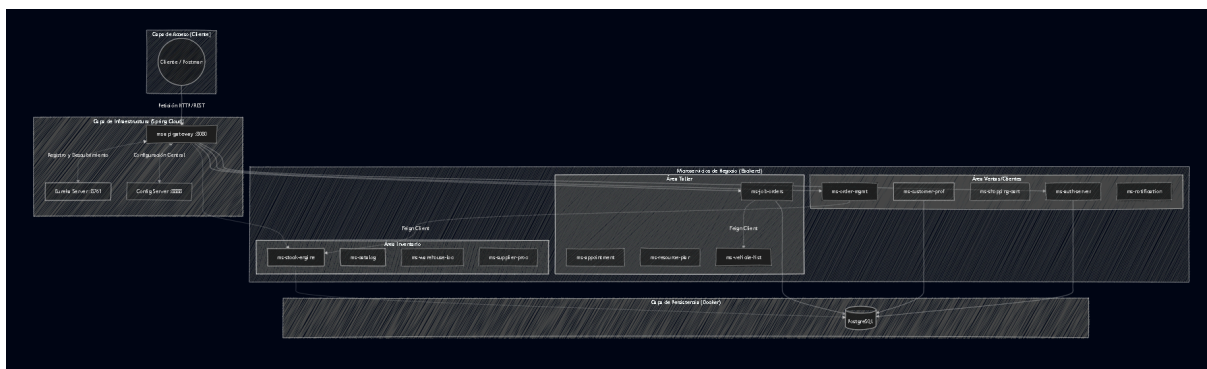
4. Arquitectura de Navegación y Flujo de Control de Acceso

Grafico frontend:



Nota: Este diagrama de flujo detalla la lógica de navegación y control de acceso de la plataforma Si el usuario inicia sesión, un servidor de autenticación (Auth-Server) filtra su acceso según tres roles específicos

Grafico backend:



Nota: Este gráfico representa la arquitectura de microservicios de tu sistema El flujo comienza con el cliente enviando peticiones REST a través de un API Gateway Los servicios de negocio están organizados modularmente en tres grandes áreas (Inventario, Taller y Ventas/Clientes) que se comunican entre sí mediante Feign Clients

4.1. Análisis del Flujo de Navegación y Seguridad

Puerta de Enlace y Autenticación: Todo acceso comienza en la *Landing Page*. El sistema evalúa inmediatamente si el usuario está logueado. En caso negativo, se restringe la navegación a la *Navegación Pública* (Home y Catálogo de Repuestos). La validación de identidad es gestionada por el **ms-auth-server**, asegurando que el acceso a datos sensibles esté protegido desde el primer contacto.

Gestión de Roles y Privacidad: Una vez autenticado, el **ms-auth-server** valida el rol específico para segmentar la experiencia y proteger la integridad de la información:

- **Rol Cliente (Portal Cliente):** Accede a funciones de consumo como el *Carrito de Compras*, *Agendamiento de Citas* y, crucialmente, a su *Perfil con Historial de Vehículo*. Esto garantiza la privacidad de los datos personales y técnicos del cliente, permitiendo que solo él visualice su información histórica.
- **Rol Mecánico (Intranet Taller):** Accede a herramientas operativas como el *Dashboard de Órdenes de Trabajo*, *Consulta de Inventario* y *Solicitud de Materiales*. Esta segmentación evita que personal operativo acceda a información financiera o datos personales no pertinentes a su labor.
- **Rol Admin (Panel de Control):** Centraliza la gestión total de la bodega, métricas de ventas y, lo más importante, la *Gestión de Usuarios y Roles*, permitiendo auditar quién tiene acceso a qué parte del sistema

4.2. Sostenibilidad y Responsabilidad Técnica

Mantenibilidad: Al separar la navegación por roles vinculados a microservicios específicos, cualquier actualización en el portal del mecánico no afecta la disponibilidad de la tienda online para el cliente.

Responsabilidad: El flujo asegura que el sistema cumpla con principios de responsabilidad ética, evitando fugas de información entre departamentos y asegurando que cada usuario interactúe únicamente con las funciones necesarias para su necesidad específica (cliente) o tarea asignada (empleado).

4.3. Justificación de Factores Críticos en el Backend

Seguridad y Privacidad: La combinación del **API Gateway** con el **ms-auth-server** asegura que solo usuarios con roles validados accedan a los microservicios de negocio. Al aislar la persistencia en contenedores Docker independientes, se garantiza que los datos sensibles del cliente en **ms-customer-prof** estén protegidos de accesos no autorizado desde otros módulos.

Sostenibilidad Técnica: El uso de **Docker y Docker Compose** permite que el sistema sea independiente de la infraestructura física, facilitando su despliegue en entornos de nube o servidores locales con un consumo de recursos optimizado. La capacidad de crecimiento (escalabilidad) está garantizada por el **Eureka Server**, que permite levantar nuevas instancias de servicios críticos ante picos de demanda.

Responsabilidad y Ética: El diseño evita el acoplamiento de datos, asegurando que la trazabilidad del vehículo (**ms-vehicle-hist**) sea un registro íntegro y auditable, cumpliendo con la necesidad del cliente de ofrecer un servicio transparente y profesional

5. Consideraciones de Seguridad, Privacidad y Sostenibilidad

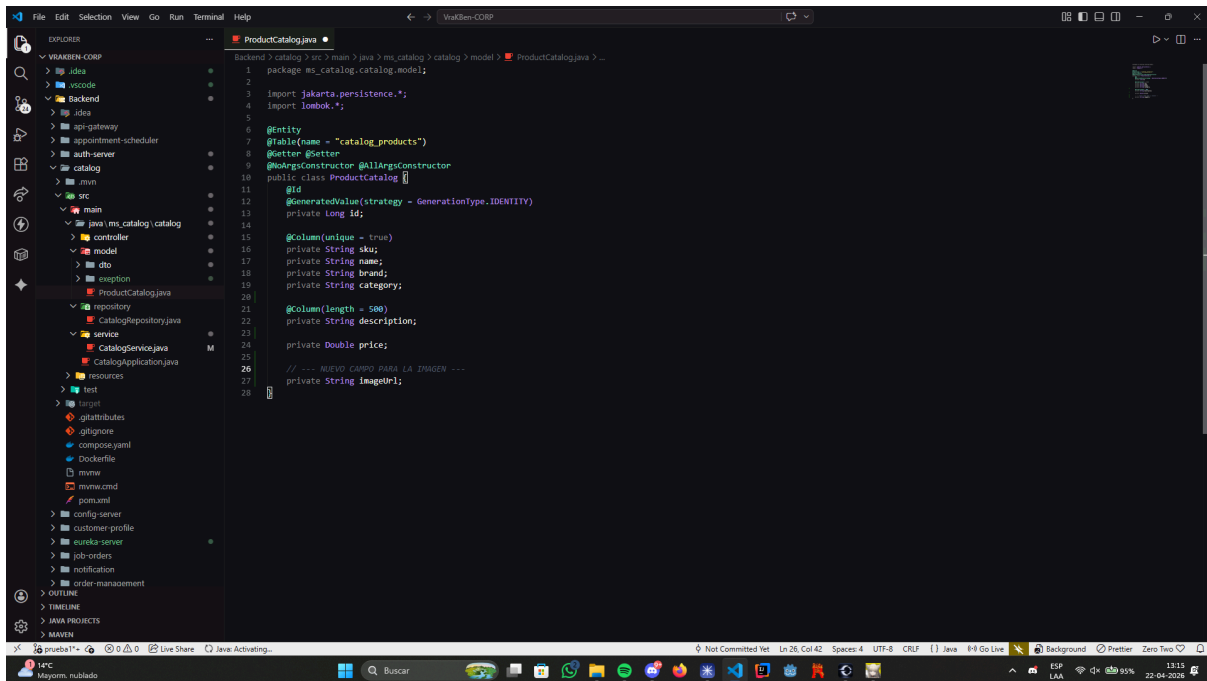
Seguridad y Privacidad: Se implementa el microservicio **auth-server** para la emisión de tokens JWT. Los datos sensibles de los clientes se aíslan en el microservicio **customer-profile**, limitando el acceso a esta información solo a servicios autorizados, cumpliendo con principios de protección de datos.

Sostenibilidad Técnica: La modularidad de los microservicios asegura que el sistema pueda evolucionar a largo plazo. Al estar desacoplados, es posible actualizar o reemplazar un módulo (ej: cambiar el proveedor de correos en **notificación**) sin reconstruir todo el sistema, reduciendo el desperdicio de recursos computacionales.

Sostenibilidad Operativa: El uso de imágenes Docker livianas (Alpine) reduce el consumo de energía y espacio en servidores, alineándose con prácticas de infraestructura sostenible.

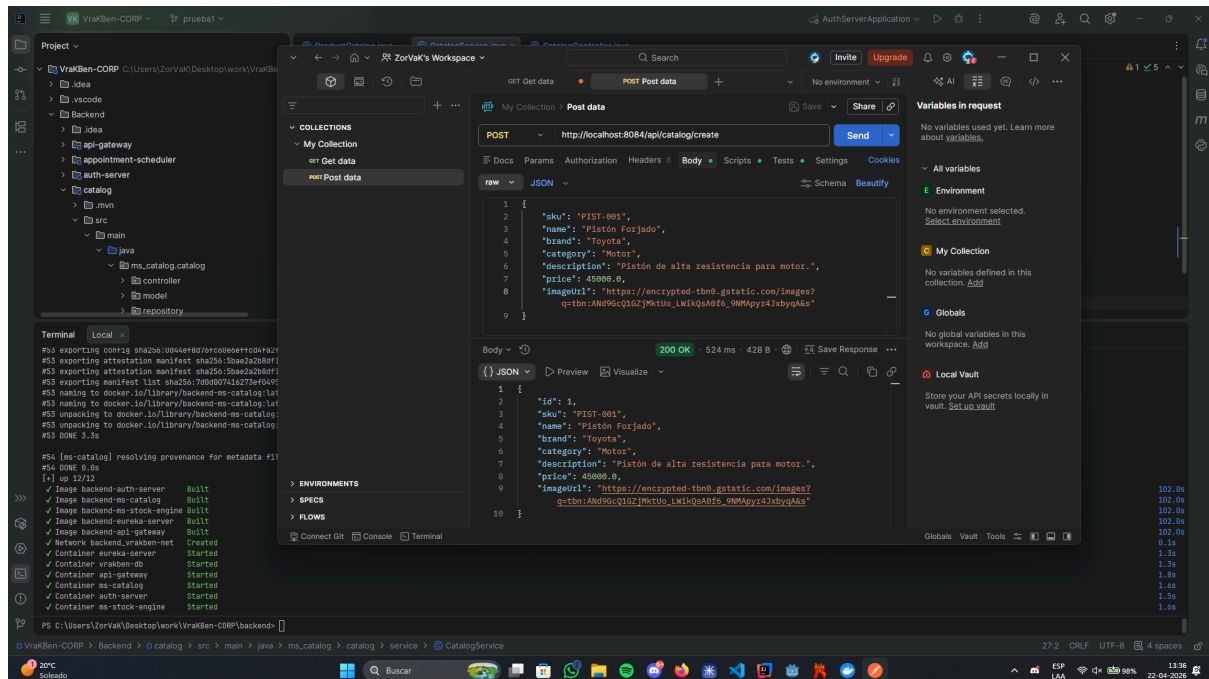
— NUEVAS COSAS

Se añadió una nueva imagen, para que en el catalog y tenga vida, ya que como ahora vamos a llevar todo a un frontend, necesitamos darle vida a los microservicios



```
1 package ms_catalog.catalog.model;
2
3 import jakarta.persistence.*;
4 import lombok.*;
5
6 @Entity
7 @Table(name = "catalog_products")
8 @Getter @Setter
9 @NoArgsConstructor @AllArgsConstructor
10 public class ProductCatalog {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @Column(unique = true)
16     private String sku;
17     private String name;
18     private String brand;
19     private String category;
20
21     @Column(length = 500)
22     private String description;
23
24     private Double price;
25
26     // ---- NUEVO CAMPO PARA LA IMAGEN ----
27     private String imageUrl;
28 }
```

– Se limpio y se volvió a construir el backend con los cambios hechos en catalog, para las imagenes y se hizo un POST para poder comprobar cambios realizados



#.\mvnw clean package -DskipTests

-(Con este comando, limpie el maven, para reiniciar en limpia la base de datos, que ya le teníamos datos puestos de la versión anterior)

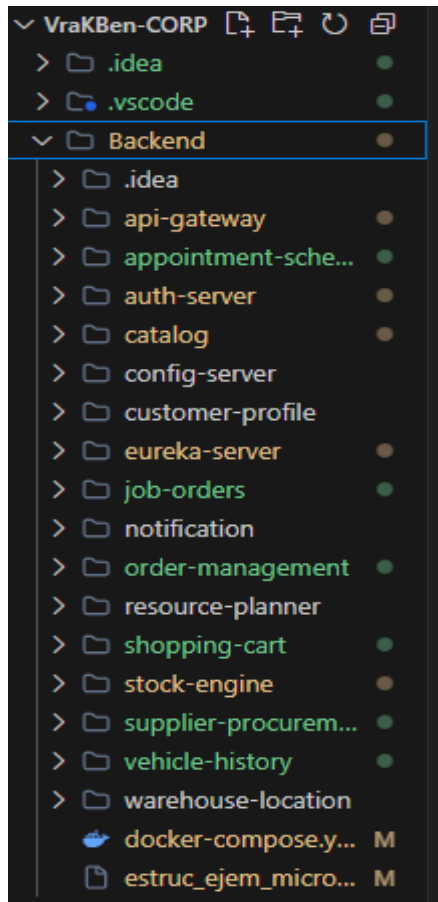
#docker-compose down

-(Con esta bajamos la base de datos de docker)

#docker-compose up --build -d

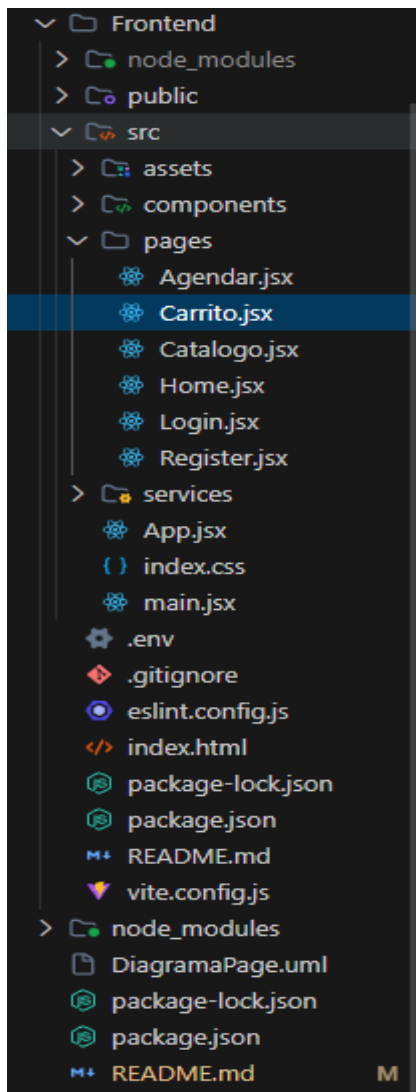
-(Con esto volvimos a construir toda la base de datos con los nuevos cambios, y ahora para crear datos bien hechos con los nuevos cambios realizados)

estructura del proyecto



Estructura del Backend

"La arquitectura del backend sigue un modelo de **Microservicios** desacoplados, contruidos a partir de **Arquetipos Maven** para garantizar la estandarización y calidad del código. La estructura organiza el sistema en componentes independientes: un **BFF (Backend For Frontend)** que actúa como intermediario optimizado para el cliente, y microservicios especializados (como **catalog** y **customer-profile**) que gestionan responsabilidades específicas del negocio. En la raíz del proyecto, el archivo **docker-compose.yml** funciona como el orquestador principal para el despliegue de contenedores, mientras que cada módulo cuenta con su propio archivo **pom.xml**, asegurando una gestión de dependencias aislada y un ciclo de vida de construcción robusto.



Estructura del Frontend

"El componente frontend del proyecto **VraKBen** se organiza bajo el estándar de gestión de **NPM**, asegurando una arquitectura modular y escalable. La estructura se divide principalmente en dos directorios clave: la carpeta **public**, que aloja el punto de entrada de la aplicación y los recursos estáticos globales, y la carpeta **src**, donde se centraliza la lógica de negocio, los componentes de la interfaz de usuario y los servicios de comunicación. Todo el ecosistema está gobernado por el archivo **package.json**, el cual define las dependencias necesarias y los scripts de automatización, permitiendo un flujo de desarrollo ágil y una integración coherente con el resto de los servicios del sistema."

